

Aswin Ram Kalugasala Moorthy

Chicago, IL · kaswinram.1603@gmail.com · +1 (872) 214-6683 github.com/Aswin-Ram-K

PROFILE

Agent-infrastructure engineer working on the layer underneath model behaviour: capability materialisation, sandboxed enforcement, multi-agent control planes, and evaluation rigs that can tell a real result from a flattering one.

Degree path that produced this: **B.Tech ECE, NIT Karnataka (2019-2023)**, then **M.A.S. ECE at Illinois Tech (Spring 2025 - Fall 2027 expected)** with graduate coursework in classical ML, computer vision and software design sitting *alongside* the systems work, not in place of it.

Currently targeting **agent-infrastructure and forward-deployed engineering roles** — summer 2027 internship now, new-graduate on graduation.

TECHNICAL FOCUS

AreaWhat I have actually built **Agent harnesses**Two kernels: a state-machine runtime where each transition re-derives its tool set (Consonance), and a control plane for designing/reviewing multi-agent workflows (Meta-Agent) **Capability security**bubblewrap -- unshare-all + explicit host allowlist; enforcement by *absence*; MCP advertisement mechanically derived from the grant set and asserted both directions **Sandboxing**unix-socket mediation between a network-less sandbox and the only broker holding credentials; per-layer mounts; kernel error codes (ENETUNREACH , ENOENT , EROFS) as the proof **Verification**35-entry offline suites where a skipped check fails; falsifiers written before the code runs; a promotion gate that fails closed when evidence refs don't resolve **Observability**433K-observation streaming ingest over 3,067 real agent sessions; append-only ledgers with _seq supersession and no delete verb **Format archaeology**Reverse-engineered a zstd multi-frame session-log format and a versioned generation-filename scheme from bytes on disk, then proved the naive read is silent-loss

SELECTED PROJECTS

1. Consonance — capability-materialisation kernel

github.com/Aswin-Ram-K/publish-present-compete/tree/main/consonance · TypeScript · AGPL-3.0 · Node ≥ 24 · 528 commits, 441 files, 16,704 LOC src/ , 18,967 LOC tests/ , 0 runtime deps

The engine has total authority and zero agency. The model has total agency and zero authority. Neither can produce a side effect alone.

Design. A state is a **continuation**: retrospective (what was decided and proven) and prospective (what may happen next) in one content-addressed object. Because truth and permission live in the same addressable thing, `diff()` can answer what no other system can — *what did this state permit that the last one did not?* — reporting **grant deltas**, not only fact deltas.

The measured claim. A live A/B ran the same task, from the same starting state, with the same model and the same prompt. The only variable was the state class:

editor (attached) reasoner (output-only) tools materialised [fs.read, fs.write] []
 workspacerwro model's planwrite module.ts write module.ts — byte-identical appliedtruefalse
 — absent: fs.write workspace digestb40dedde → aef2e4ab b40dedde — **unchanged**
 network BLOCKED: ENETUNREACH BLOCKED: ENETUNREACH reasoning tokens6331166
 The model wanted to write the file in both runs. In one it could; in the other the capability did not exist — **not refused, absent**. 100 % the class, 0 % the prompt.

Boundary. bwrap --unshare-all with an explicit host allowlist (the Node prefix, /usr, the lib dirs, /etc/{passwd,nsswitch.conf}, /dev/{null,urandom,zero}) — not a blanket --ro-bind / /. The sandbox holds **no IP network**; it reaches a broker over a unix socket, and the broker is the only holder of the model endpoint. An ungranted model (kimi-k3) is refused by the broker; a detached layer has layer.broker === null, so reaching for it is a language-level TypeError.

The gap I closed myself. E10-1: buildArgv() used to open with --ro-bind / / — the entire host, read-only, visible to every sandbox; a probe could stat /root, /etc/shadow, all of /home. Replaced with the allowlist; the same probe now returns ENOENT for all three. Logged as backlog item **E10-1**, done.

Verification. 35 entries (34 suites + 8 evals), and a skipped check counts as a failure — "a suite that silently skips is worse than no suite." The MCP advertisement is asserted to equal the grant set **two ways**: advertised ⊆ granted and granted ⊆ advertised, plus a named ungranted tool asserted **absent** rather than present-and-refused. The conformance stub suite includes a SUPERSET case that *must fail*, so the suite can catch its own drift.

Two real bugs, found by running: a 90 s askBroker timeout never cleared on success (every step paid dead wait *after* the answer arrived), and empty completions returned as ok: true, text: "" whenever a reasoning model spent its budget on reasoning_content. Both recorded as D-021.

Governance. 73 decisions in an adversarial log (D-001+), each with the alternatives rejected; 14 epics / 99 work items in BACKLOG.md with acceptance criteria; 11 pre-registered experiments with falsifiers written before the code ran. 8 EMERGE, 2 CLOSE, 1 awaiting budget. Three types graduated tools/ → src/ while **no engine moved** — promoting a type is not promoting a mechanism.

The honest part: of the 11 experiments, one attractive result — a 53.8 % token saving at identical success — was **voided by its own pre-registered control**. It is published as void, not quietly dropped.

2. cockpit — observability and signal pipeline

publish-present-compete/app · TypeScript · 112 tests, 6,032 LOC, 3 commits (not yet public)

A standalone Cordis app that ingests a real agent-usage corpus and turns work + interests into research and tickets. Built as a product, not a demo.

- **Scale measured:** 433,269 observations · 3,067 sessions · 190,865 steps · 230,635 tool calls · 8,654 turns · billable 22.59B tokens (5.7 % cache reuse). 100 % census coverage.
- **Streaming ingest.** The first version OOM'd at ~4 GiB on 2,982 logs. Fixed by processing one session at a time with bounded caches and a persisted file index: peak RSS 9.3 GiB for a 329 MiB store. Also worked around a Node zstdDecompressSync native-context leak by recycling worker processes (~20 files per child — the full scan then takes 13 s instead of never finishing).
- **Format archaeology:** a session log is *concatenated zstd frames*, so a naive single-frame decompress silently yields **one** record. Reproduced in a 3-line test that runs in CI.

- **Four defects caught by running it against the corpus, all of which passed a green unit suite:**
- every tool result was unpaired — a real `tool/result` carries **no** `callId`, so 17,932 calls read as "not completed";
- `capture.health()` summed tokens on **every rung**, triple-counting roll-ups (66.4B vs 22.1B);
- the promotion gate **passed by default** when evidence refs did not resolve — now fails closed;
- two confounder axes read the wrong path (`effort` populated 0 % against a measured 61.85 %; `skill_catalog_set` had 1 distinct value where the corpus has 7).
- The lesson, recorded in the code: *unit tests prove the adapter does what was written; only the corpus proves what was written matches reality.*
- **Provenance:** 131 claims in a TSV ledger, each carrying a falsifier; a claim without one **throws**. 82 sources. The ledger validator caught 6 real defects during construction, five of them the same class — a claim resting on a source that was never fetched.

3. Meta-Agent — review-first control plane

Private — MIT-licensed · source walkthrough on request · Python · MIT · FastAPI + PostgreSQL + Temporal + React

A self-hosted control plane for designing, observing and **reviewing** multi-agent workflows: one place for conversations, reusable roles, conditional workflow graphs, durable runs, sandbox artifacts, and an explicit promotion decision.

Built as a transparent foundation for human-in-the-loop operation — **not** a black-box autonomous coding system. The README carries a "what is not implemented yet" list (auth, RBAC, real tool execution, writing approved diffs back to source) because the alternative is a stranger discovering it from a security incident instead of from the front door.

4. Valhalla — local-first agent platform

`private` · TypeScript · *~233 commits · no LICENSE file yet (README badge is decorative)*

A local-first autonomous-agent platform built from scratch on a minimal substrate: a lightweight composer reads the recent session tail plus relevant memory each turn and assembles a **bounded relevance bundle**; the main model reasons over exactly that. Expensive context stays small, the cache stays stable, and most token traffic never leaves local hardware. Self-hosted eval harness over function-calling, run against local models.

5. Alfred — autonomous coursework assistant

`private` · JavaScript · *244 tracked files*

Pulls an assignment from Canvas, reads the course's own material, solves the stated problems, verifies answers against that material, renders the submission PDF, then **waits for one human approval** before anything is uploaded or pushed.

All model traffic runs on one lane with sticky failover rotation and a rate ledger; an unconfigured or unhealthy lane **fails closed** — no request is made, and the run says why.

6. WEBSEER — meta-search design study

`private` · Rust · *115 tracked files*

Research-driven design: a 10,080-word dissection of SearXNG internals — pipeline, engine-plugin model, the exact ranking function, dedup, limiter, API surface, observability,

governance, and 15 specific exploitable weaknesses; a 6-family competitive survey; six locked architecture-decision records (router-first learned routing; the router-biases/scorer-decides boundary; the polyglot Python-model / Rust-runtime split); and the measured economics of owning a search index.

7. ticket-fleet — fleet operating model

private · JavaScript · 88 tracked files

An operating model for running an autonomous agent fleet against a git repository where **the issue tracker is simultaneously the work queue and the control plane, no agent ever validates its own work**, and a user-only control channel can start or stop every part of the fleet without killing the harness.

APPLIED MACHINE LEARNING (GRADUATE COURSEWORK + EARLIER)

ECE 563 — AI in Smart Grid (IIT): five-project supervised-learning sequence — linear/logistic regression, decision trees, SVM, kNN, MLP, random forest + gradient boosting with GridSearchCV 5-fold CV, and Lasso/Ridge feature selection on PMU smart-grid time-series.

ECE 565 — Computer Vision & Image Processing (IIT): Gonzalez & Woods text; spatial and frequency-domain filtering, segmentation, feature extraction. Programmatic work on iterative histogram thresholding (§10.3.2) and chain-code boundary representation with starting-point invariance (NumPy / SciPy / PIL / Pillow).

ECE 448/528 — Application Software Design (IIT): Java + IoT REST API extending an MQTT IoT Hub (Eclipse Paho); 5 endpoints, red-green TDD, integration testing against a simulator and a grading harness.

Computer Vision & Malware Detection (2022-23): CNN + LSTM hybrid on Microsoft Maling (opcodes as text → LSTM; binaries as images → CNN), plus a Keras/PyTorch GAN augmenting under-represented malware-family classes. Reported **98.37 % accuracy / 95.91 % precision / 95.22 % recall** held-out.

CNN for retinal fundus disease detection (2021): 45+ ophthalmic classes, custom augmentation pipeline, >75 % validation accuracy.

Analog front-end amplifier (2021): capacitive-feedback OTA + MOSFET pseudo-resistors tuned for low input-referred noise, for low-amplitude neural signal acquisition.

PROFESSIONAL EXPERIENCE

Business Development Executive — SkilloVilla · Oct 2023 - Feb 2024 Ed-tech sales conversations end to end; top-performing team on the floor. Non-engineering, included for continuity and because the communication discipline transfers directly to a forward-deployed seat.

PRACTICES I ACTUALLY KEEP

- **A claim without a falsifier does not ship.** The 131-claim ledger in the cockpit enforces this in code — constructing a claim with no falsifier throws.
- **Nothing is deleted; it is superseded.** Append-only ledgers with bi-temporal `valid_at/invalid_at`, and no delete verb in the store.
- **Pre-registration before measurement.** A finding may only be promoted if it was sealed before the confirming data existed. Exploratory findings can never be promoted.

- **Volume is a failure mode.** Promoted findings are capped at three.
- **Measure before believing a number** — including my own. Five of my previous claims were falsified by measurement in a single review pass and corrected in place.

CONTACT

kaswinram.1603@gmail.com · +1 (872) 214-6683 · github.com/Aswin-Ram-K · [publish-present-compete](#)

Work authorization: F-1 student authorization. Seeking summer 2027 internship and new-graduate roles.